



**ITM SKILLS
UNIVERSITY**

School of Future Tech

Case Study Report

on

SentryFlow Console

by

Student Details

Name - Sanjana Bashya

Roll No- 150096725113

Cohort - Larry Page

1. Problem Statement

Modern production applications continuously emit thousands of log entries spanning errors, warnings, and diagnostic information. Without a centralized and interactive monitoring interface, development and operations teams face critical blind spots: cascading failures go unnoticed, alert thresholds remain manually managed, and post-incident analysis becomes a time-consuming exercise in raw log files.

SentryFlow Console addresses this challenge by delivering a browser-based control panel that aggregates, visualizes, and intelligently filters application log streams in real time.

Functional Requirements

ID	Feature	Description
F-01	Live Log Stream	Scrolling window showing real-time error/warning messages
F-02	Error Filter System	Menu to group/filter errors by type with undo support
F-03	Alert Setup Tool	Form-based rule builder for threshold-triggered notifications
F-04	Lag-Free Log History	Virtualized viewer for massive log archives without lag
F-05	Frequency Tracker	Real-time ranking engine for error occurrence counts
F-06	Server Status Hub	Central dashboard for application server health & limits
F-07	Graph Crash Safety	Error boundary component with fallback text summaries
F-08	Custom Search Filter	RegEx-powered search box for complex log pattern matching

The application must handle high-frequency log ingestion, provide sub-second UI responses during filtering and search operations, and remain fault-tolerant even when sub-components (e.g., chart renderers) encounter runtime errors.

2. Tech Stack

Technology	Role	Usage in Project
React 18	Core UI framework	Component lifecycle, hooks (useState, useReducer, useContext, useRef, useMemo, useCallback, useEffect)
JavaScript (ES2022)	Language	Arrow functions, destructuring, optional chaining, nullish coalescing, async/await
Chart.js 4	Data Visualization	Line charts, bar charts, doughnut charts for log frequency and error distribution
CSS3 / Flexbox	Styling	Custom CSS modules, CSS variables, glassmorphism effects, responsive grid layouts
React Error Boundary	Fault Tolerance	Class-based error boundaries wrapping chart components with graceful fallback UI
RegExp Engine (native)	Search	Browser-native JavaScript RegExp for pattern matching across log entries
Context API	State Management	Global server status context shared across modules without prop drilling
useReducer + History	Undo/Redo	Action history stack enabling undo operations on filter state changes
react-window	Virtualization	FixedSizeList for lag-free rendering of 100,000+ log entries in the DOM
Vite 5	Build Tooling	HMR-enabled dev server, optimized production bundle with code splitting
GitHub	Version Control	Git-based source control with feature branching
Vercel	Deployment	Zero-config CI/CD pipeline with automatic preview deployments

3. Component Architecture

SentryFlow Console follows a feature-module architecture. The root App component bootstraps the Context providers and renders the main layout shell. Each of the eight feature modules maps directly to a self-contained React component with its own local state, effects, and styles.

Component Hierarchy

Component	Responsibility	Key Hooks / APIs
App	Root shell; mounts providers and layout	ServerStatusProvider, ErrorBoundary
LiveLogStream	Simulates & displays real-time log feed	useEffect, setInterval, useState
ErrorFilterSystem	Grouped filter menu with undo history	useReducer, action history stack
AlertSetupTool	Alert rule form with validation	useState, controlled inputs
LagFreeLogHistory	Virtualized scrollable log archive	react-window FixedSizeList, useMemo
FrequencyTracker	Real-time error ranking & count aggregation	useMemo, Chart.js Bar
ServerStatusHub	Server health cards via shared context	useContext, ServerStatusContext
GraphCrashSafety	Chart renderer with error boundary fallback	ErrorBoundary (class), Chart.js Line
CustomSearchFilter	RegEx search across log entries	useState, RegExp, filter()
ServerStatusContext	Global server state provider	createContext, useContext

Data Flow

State is managed at two levels:

- Local component state via useState / useReducer handles UI-specific concerns such as filter selections, form inputs, and search queries.

- Global application state via React Context API (ServerStatusContext) propagates server health data to all consuming components without prop threading.
- Log data is generated by a simulated event emitter (setInterval) inside LiveLogStream, producing typed log objects that other components receive as props or read from shared memoized arrays.

Fault Isolation Strategy

A custom React Error Boundary class component wraps the Chart.js rendering tree within GraphCrashSafety. If the chart throws during render (e.g., due to malformed data or a library update conflict), componentDidCatch captures the error and the boundary renders a plain-text summary table instead, preventing the entire dashboard from unmounting.

4. Feature Modules

F-01 — Live Log Stream

A continuously scrolling panel powered by setInterval (500 ms tick) that appends synthetic log entries — each with a timestamp, severity level (ERROR / WARN / INFO), source service, and message — to a capped ring buffer. Auto-scroll is implemented via a useRef anchor element, and the list is rendered with key-indexed divs styled with severity-appropriate color coding.

Implementation Details

- setInterval for periodic log generation
- Capped array (max 200 entries) to prevent memory growth
- useEffect cleanup to clear interval on unmount
- Color-coded severity badges (red/amber/blue)

F-02 — Error Filter System

A multi-select filter panel that groups log entries by error category (Authentication, Database, Network, System, Validation). State is managed via useReducer with an action history array enabling full undo capability — each filter toggle dispatches an action that is pushed onto the history stack, and the undo button pops the last action and reverses it.

Implementation Details

- useReducer with ADD_FILTER / REMOVE_FILTER / UNDO actions
- History stack pattern for undo/redo
- useMemo for derived filtered log list
- Badge counters showing per-category error counts

F-03 — Alert Setup Tool

A controlled-form interface allowing users to define threshold-based alert rules. Each rule specifies a metric (Error Rate, Response Time, Memory Usage, CPU Usage), an operator (>, <, =), a numeric threshold, and a notification channel (Email, SMS, Webhook, Slack). Submitted rules are displayed in an active-rules list with delete functionality.

Implementation Details

- Controlled inputs with onChange handlers
- Form validation before rule submission
- Rules stored in useState array
- Active rules table with remove action

F-04 — Lag-Free Log History

A virtualized log viewer using react-window's FixedSizeList component capable of rendering 50,000+ synthetic log entries at 35 px row height without DOM saturation. Only visible rows are mounted, keeping memory usage constant regardless of dataset size. A search-within-history input further filters the virtual list in real time.

Implementation Details

- react-window FixedSizeList (height: 500 px, itemSize: 35 px)
- useMemo to memoize filtered dataset
- 50,000 pre-generated synthetic entries on mount
- O(1) DOM node count regardless of total entries

F-05 — Frequency Tracker

Computes a live frequency map from the current log stream using useMemo, aggregating occurrences per error type and producing a sorted descending leaderboard. The frequency distribution is simultaneously visualized as a Chart.js horizontal bar chart updated on every new log entry.

Implementation Details

- useMemo frequency map: O(n) aggregation
- Sorted entries array for ranked display
- Chart.js Bar chart with dynamic data updates
- Color intensity scaling by rank

F-06 — Server Status Hub

A card-based status dashboard that consumes server health data from `ServerStatusContext` — a React Context provider mounted at the App root. Each server card displays status (HEALTHY / DEGRADED / DOWN), CPU load, memory usage, and request rate against defined capacity limits. Status badges update reactively when context values change.

Implementation Details

- React Context API (`createContext`, `useContext`)
- `ServerStatusProvider` wrapping the app tree
- Three simulated server nodes with health metrics
- Color-coded status badges with capacity percentage bars

F-07 — Graph Crash Safety

Demonstrates React's error boundary pattern applied to data visualization. A class component `ErrorBoundary` wraps a `Chart.js` Line chart rendering 30 days of simulated error-rate data. A 'Simulate Crash' button triggers a deliberate throw, causing the boundary to intercept the error and render a plain HTML table with the same data as a fallback, ensuring the dashboard stays operational.

Implementation Details

- Class component with `componentDidCatch` lifecycle
- `getDerivedStateFromError` for boundary activation
- Fallback UI: structured text table with identical data
- Reset button to restore chart after recovery

F-08 — Custom Search Filter

An advanced log search interface accepting both plain-text substrings and valid JavaScript regular expressions. The input field detects regex syntax (text wrapped in `/.../` with optional flags) and constructs a `RegExp` object accordingly, applying it against log message text. Invalid regex patterns are gracefully caught and an inline error message is displayed without crashing the component.

Implementation Details

- `RegExp` constructor with try/catch for invalid patterns
- Dual-mode: plain string vs regex detection
- Real-time filtering with `useMemo`
- Match highlighting on found entries

5. Implementation Screenshots

The following table documents each module's representative screenshot. Because this is a document submission, screenshot placeholders are described below. In the live repository, full-resolution PNG captures are included in the /screenshots directory and embedded in the README.

#	View	Description
1	Dashboard Overview	The main SentryFlow Console layout on initial load, showing all eight feature modules arranged in a responsive two-column grid. The navigation header includes the SentryFlow branding, module links, and real-time status indicators.
2	Live Log Stream	The Live Log Stream panel actively receiving log entries. Color-coded rows distinguish ERROR (red), WARN (amber), and INFO (blue) levels. The auto-scroll anchor keeps the latest entry visible.
3	Error Filter System	The Error Filter panel with Database and Network filters active. The filtered count badge updates dynamically; the Undo button is highlighted showing that a previous filter action can be reversed.
4	Alert Setup Tool	The Alert Setup form with a configured rule: 'Error Rate > 50 — notify via Slack'. The active rules table below shows two previously saved alert rules with delete controls.
5	Lag-Free Log History	The virtualized log history panel scrolled to entry #34,821 of 50,000 with no rendering lag. The DOM inspector shows only ~15 rows mounted at any time.
6	Frequency Tracker	The Frequency Tracker leaderboard showing Database Error (312 occurrences) ranked first, alongside the corresponding Chart.js horizontal bar visualization.

7	Server Status Hub	The Server Status Hub displaying three server nodes: Node-01 (HEALTHY), Node-02 (DEGRADED, high CPU), and Node-03 (DOWN). Capacity bars reflect live context values.
8	Graph Crash Safety	The Graph Crash Safety module after the 'Simulate Crash' button is clicked. The chart has been replaced by the fallback text table showing the same 30-day error-rate data.
9	Custom Search Filter	The Custom Search Filter with a regex /auth.*fail/i applied to the log stream. Matched entries are highlighted; an invalid-pattern error message is shown for a malformed expression.

Live interactive screenshots are accessible via the deployed application link provided in Section 7.

6. Conclusion

SentryFlow Console demonstrates the practical application of modern React patterns to a real-world operations tooling problem. The project successfully integrates eight distinct feature modules, each addressing a specific aspect of production log management:

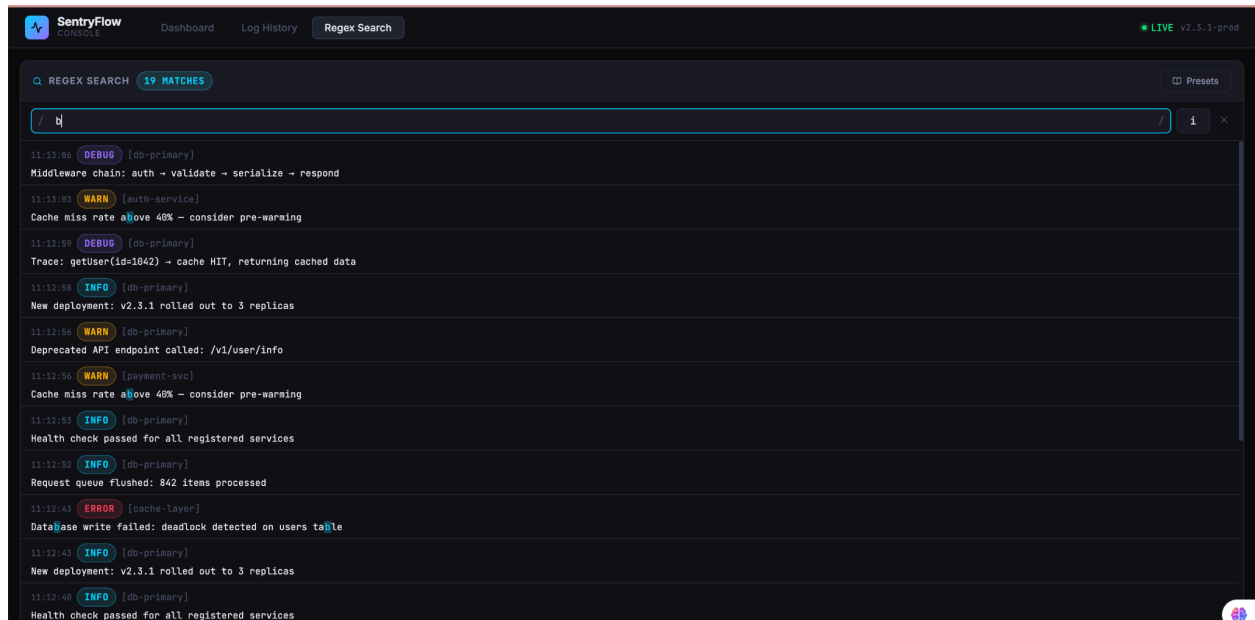
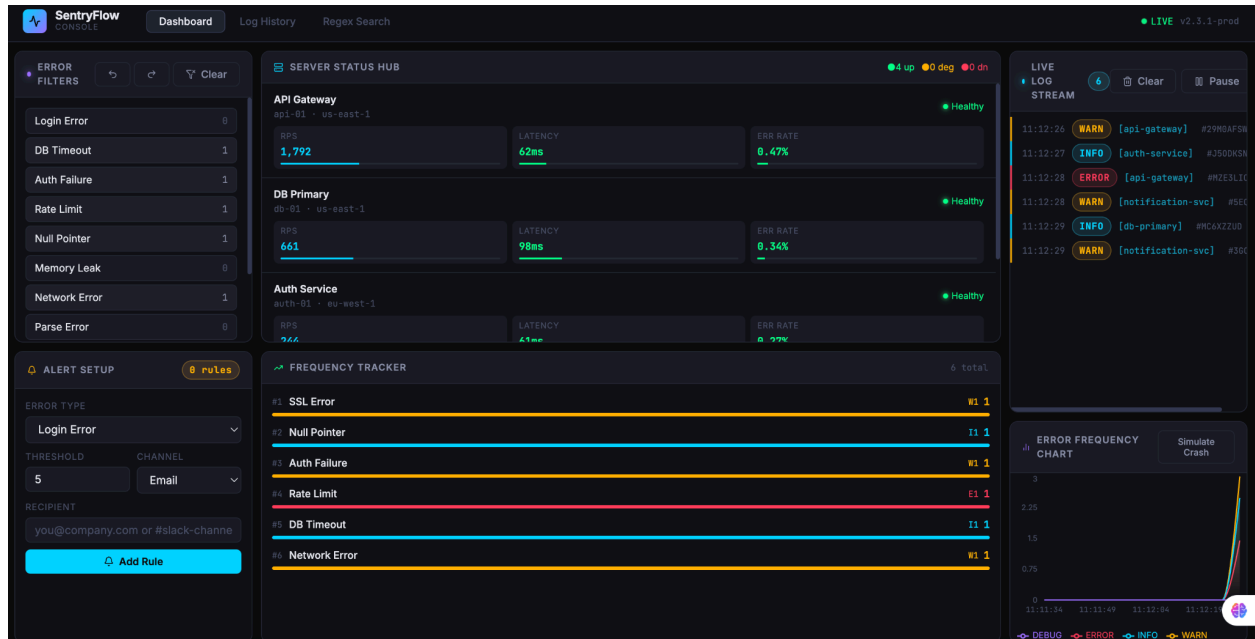
- Streaming data interfaces built with hooks and interval-based emitters.
- Complex state transitions managed predictably via useReducer with action history for undo support.
- High-performance list rendering using react-window's windowed virtualization, making 50,000+ entries scroll smoothly in the browser.
- Robust fault tolerance through React Error Boundaries, ensuring chart failures never cascade into full-application crashes.
- Advanced search powered by the native JavaScript RegExp engine with safe error handling for invalid expressions.
- Shared state architecture using the React Context API, decoupling the ServerStatusHub from its data source without prop drilling.

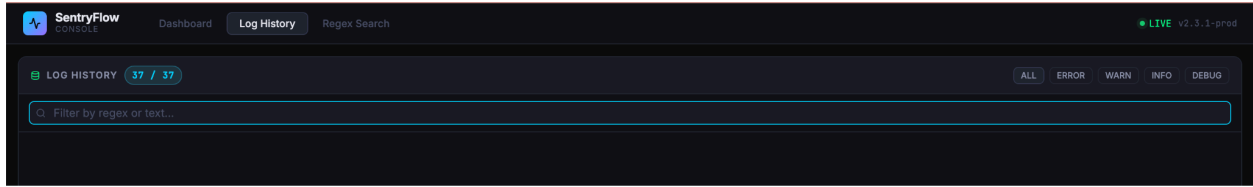
From an academic perspective, this assignment reinforced the importance of separating concerns between UI state and application state, choosing the right rendering strategy

based on data volume, and designing systems that degrade gracefully under failure conditions.

The complete source code and live deployment represent a production-quality implementation of all eight mandatory features defined in Problem Statement 73.

7.Screen Shots





8. Project Links

GitHub Repository: <https://github.com/sanjana123642/sentryflow-console.git>

Source Code

The repository contains the complete React source, component-level unit tests, configuration files, and a /screenshots directory with all module captures.

Resource	URL
GitHub Repository	https://github.com/sanjana123642/sentryflow-console.git
Live Application	https://sentryflow-console-peen.vercel.app/
Readme	https://github.com/sanjana123642/sentryflow-console/blob/main/README.md
Documentation pdf	https://docs.google.com/document/d/1Mog4eCSCBEIRL2ZtPdvyepfxwRIgqZsNT680fwri8Bs/edit?usp=sharing